

**KWAME NKRUMAH UNIVERSITY OF SCIENCE AND TECHNOLOGY
OBUASI CAMPUS**



**COLLEGE OF ENGINEERING
DEPARTMENT OF CIVIL ENGINEERING
CE 257, COMPUTER PROGRAMMING**

**APPLICATION FOR ANALYZING CONTINUOUS
BEAMS**

GROUP MEMBERS:

1. Kumi Rich-love Abena- 3949524
2. Barbara Oforiwaa Berko - 3948524
3. Akagyiirim Petrina-3947524

Date: May,2026

TABLE OF CONTENTS

Section Description	Page Number
1.0 Introduction	3
2.0 Objectives	3
3.0 Methodology	3
4.0 Tools and Technologies	4
5.0 Project Description	4
6.0 Implementation & Worked Example	5
7.0 Challenges	6
8.0 Conclusion	7

1.0 INTRODUCTION

This project aims to build a Python-based application that can efficiently solve multi-span continuous structural beams under variable load distribution types. The solution is tailored to meet the structural design verification needs of KNUST civil engineering students by automating the process of evaluating indeterminate structures using a system of linear simultaneous equation matrices based on the classical Theorem of Three Moments.

2.0 OBJECTIVES

- To automate the calculation and graphical structural tracking of indeterminate beams using Python.
- To allow the system to read customized geometry spans alongside loading layouts and generate analytical system matrices instantly.
- To generate an accurate summary of engineering outputs including continuous support moments, reaction coordinates, and vector line diagrams.

3.0 METHODOLOGY

- The system was developed using Python, leveraging standard scientific frameworks including NumPy for handling array systems and Tkinter for generating clean graphical workspace windows.
- A parameterized mathematical verification scheme was compiled directly into the application classes to process arbitrary geometry boundaries.
- The application accepts localized variables from user fields, sets individual clamping moment matrices via rigid segment algorithms, and executes simultaneous array solutions.
- It then resolves complete system reactions and outputs numerical values to custom display canvas tabs.

4.0 TOOLS AND TECHNOLOGIES

- **Programming Language:** Python
- **Libraries & Frameworks:**
 - NumPy - Numerical array tracking and system linear solvers (`np.linalg.solve`).
 - Matplotlib - Full-density structural coordinate grid graphing for SFD and BMD layouts.
 - Tkinter / ttk - Component widget window rendering and graphical user interface notebook management.
- **Platform:** Desktop standalone architecture (Cross-platform capability).

5.0 PROJECT DESCRIPTION

This project involved developing a multi-span structural desktop helper using Python to solve continuous elastic beams under point loads and uniformly distributed loads (UDL). The application is designed to ingest multi-span parameters, detect constraint conditions at boundaries, process matrix elements under simultaneous actions, and calculate final continuous profiles. The ultimate goal is to enhance calculation accuracy and analysis speeds within structural environments like KNUST.

6.0 IMPLEMENTATION & WORKED EXAMPLE

Tools Used: Python, NumPy (for array solving computations), Tkinter (for interface windows), Matplotlib (for analytical vector curves).

6.1 System Logic Flow Steps

1. Define beam geometry variables using BeamGeometry to specify lengths across multiple continuous spans.
2. Capture joint settings with SupportConditions and record environmental forces using LoadApplication entries.
3. Call the ThreeMomentSolver loop matrix solver engine to evaluate fixed-end moment thresholds and execute structural equations.
4. Extract continuous values into ShearForceCalculator to balance free bodies and build coordinates for final diagrams.

6.2 Sample Application Verification Problem

To verify the accuracy of the underlying script formulas, consider a 3-span continuous concrete structure initialized under asymmetric parameters matching the source default configuration requirements: Lengths $L1 = 5.0$ m, $L2 = 5.0$ m, and $L3 = 5.0$ m over simple outer constraints ($MA = MD = 0$).

Loading Conditions: Span 2 is loaded with a full Uniformly Distributed Load (UDL) of intensity $w = 12$ kN/m, and Span 3 carries a single concentrated mid-span point load $P = 40$ kN. Span 1 remains completely clear of external forces.

6.3 Step-by-Step Code Execution Mathematics

1. Fixed-End Moments (FEM):

- Span 1: $FEM_{AB} = 0$, $FEM_{BA} = 0$
- Span 2: $FEM_{BC} = FEM_{CB} = (w * L2^2) / 12 = (12 * 5^2) / 12 = 25.000$ kN·m
- Span 3: $FEM_{CD} = FEM_{DC} = (P * L3) / 8 = (40 * 5) / 8 = 25.000$ kN·m

2. Assembly of Simultaneous Continuity Rows:

At Node B: $2 * (L1 + L2) * MB + L2 * MC = -6 * (FEM_{BA} + FEM_{BC})$ ↳ $2 * (5 + 5) * MB + 5 * MC = -6 * (0 + 25.0)$ $20 * MB + 5 * MC = -150$
At Node C: $L2 * MB + 2 * (L2 + L3) * MC = -6 * (FEM_{CB} + FEM_{CD})$ ↳ $5 * MB + 20 * MC = -300$ $+ 2 * (5 + 5) * MC = -6 * (25.0 + 25.0)$

3. Matrix Solver Execution (handled via np.linalg.solve()):

Matrix Equation Form: A * M = B

[20 5] [MB]

=

[-150]

[5 20] [MC]

[-300]

Solving this simultaneous matrix returns the exact internal support moments:

- MA = 0.000 kN·m
- MB = -4.000 kN·m
- MC = -14.000 kN·m
- MD = 0.000 kN·m

6.4 Resolved Support Reactions & Equilibrium Verification

By executing segment balancing routines, the application isolates each span to determine the vertical upward forces acting at each support:

Support Joint	Global Beam Coord (m)	Computed Continuity Moment (kN·m)	Upward Support Reaction Force (kN)
Support A	0.0 m	0.000	-0.800 kN (Uplift Action)
Support B	5.0 m	-4.000	32.800 kN
Support C	10.0 m	-14.000	49.200 kN
Support D	15.0 m	0.000	18.800 kN

Equilibrium Check Calculations:

- Total Downward Loads: (12 kN/m * 5.0 m) + 40 kN = 100.000 kN
- Total Upward Reactions: -0.800 + 32.800 + 49.200 + 18.800 = 100.000 kN
- Numerical Closure Discrepancy Margin: 0.000000 kN (Perfect structural equilibrium closure verified).

7.0 CHALLENGES

- The system faced difficulty parsing singular matrices if overlapping zero-length spans or duplicate support boundary entries were supplied by users.
- Due to limitations in processing dynamic inputs within separate loops, text truncation issues sometimes occurred when switching canvas frames rapidly.

- Improper notation formatting or missing text inputs occasionally interrupted runtime solvers, requiring explicit try-except assertion safety layers.
- Limited execution window schedules and few verification load cases restricted advanced secondary sorting and validation procedures.

8.0 CONCLUSION

This project successfully developed a standalone structural engineering application that creates mathematical layouts from user variables and implements matrix arrays to automatically calculate multi-span beam continuous states. Despite specific entry constraints, the system demonstrates strong potential to improve calculation accuracy and validation processing speeds for KNUST's civil engineering workflows. Further UI enhancements and additional configuration options will help refine the application for practical use.